Annex I
Protocol Enforcement Mechanics
BEAP™ / WR Desk™ Canonical Specification

I.0 Status, Scope, and Relationship to Canon

Status

This Annex is **normative** unless explicitly marked as informative.
It defines mandatory enforcement mechanics, architectural constraints, operational rules, and interaction boundaries for BEAP™-conformant implementations.

Scope Boundary

This Annex does **not** redefine or extend:

- Wire formats
- Capsule serialization formats
- Cryptographic primitives
- Transport mechanisms
- PoAE™ execution semantics

These elements remain exclusively defined in the Canon and its Normative Annex.

This Annex defines **how enforcement is implemented** at runtime and architectural level.

Relationship to the Canon

- The Canon defines **what** is required.
- Annex I defines **how** those requirements are enforced.
- Nothing in this Annex weakens or contradicts Canon requirements.

I.1 Architectural Separation Principles

I.1.1 Separation of Probabilistic Reasoning and Governance Enforcement

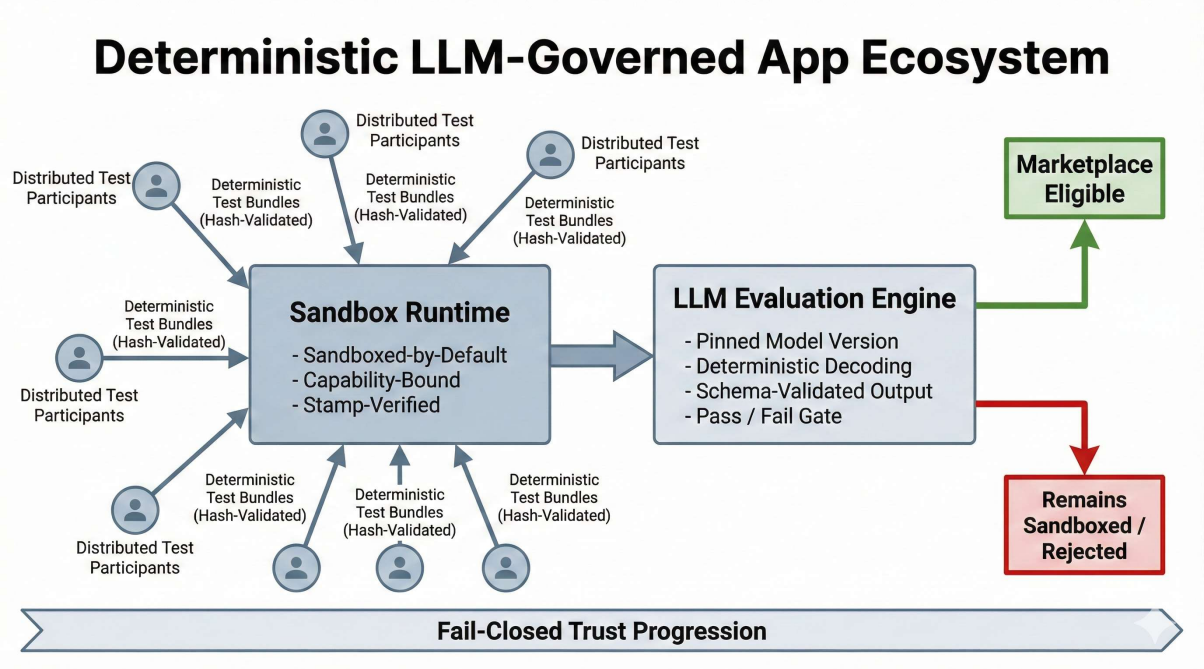
The system SHALL maintain strict architectural separation between:

- Probabilistic reasoning components (e.g., LWM / Optimando)
- Deterministic governance enforcement layers (capability engine, policy engine, PoAE™ validation, attestation enforcement)

The LWM:

- MAY propose actions
- MAY propose workflow transitions
- MAY request tool invocations
- SHALL NOT execute tools directly
- SHALL NOT elevate privileges
- SHALL NOT modify capability scopes
- SHALL NOT override policy decisions

All execution authority SHALL remain external to the model.



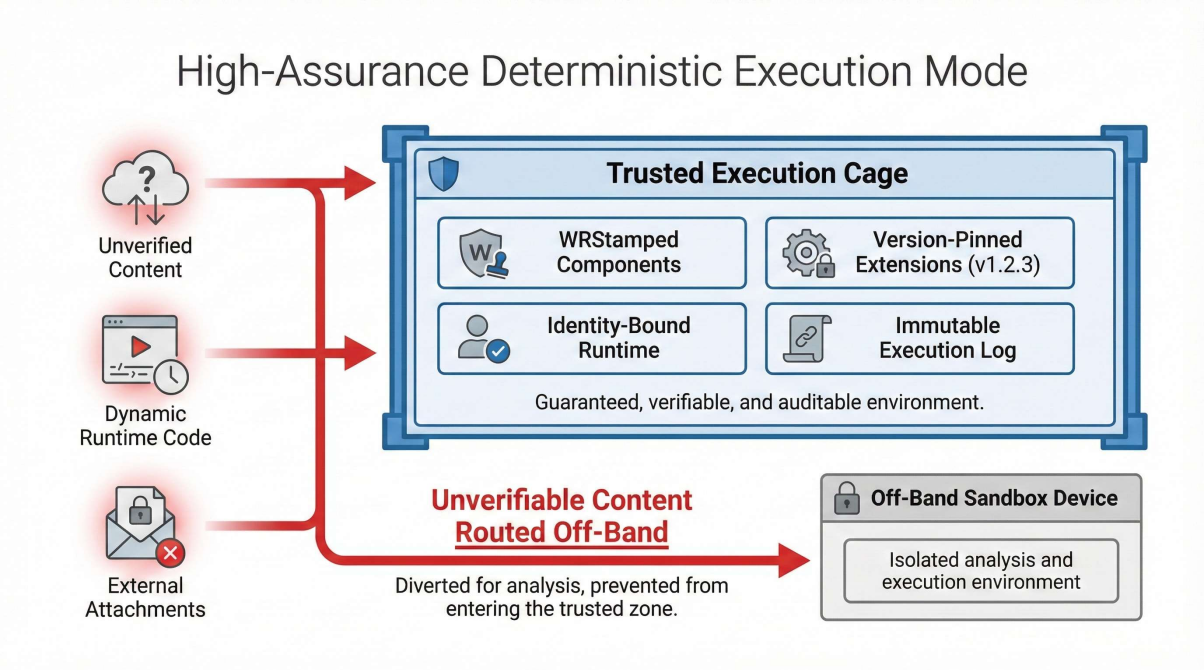
I.2 Mandatory Sub-Orchestrator Architecture

I.2.1 Primary and Sandbox Sub-Orchestrators (Normative)

Every conforming WR Desk™ implementation SHALL include:

- One Primary (Host) Orchestrator
- At least one Sandbox Sub-Orchestrator

The sandbox sub-orchestrator is a **mandatory** component and SHALL NOT be optional.



I.2.2 Mandatory Sandbox Execution Boundary

The following SHALL execute exclusively inside the sandbox sub-orchestrator:

- Sandboxed applications and extensions

- Unverified or newly ingested artifacts
- Workflow capsules prior to trust escalation
- External-origin artefacts (emails, attachments, third-party content)

The sandbox sub-orchestrator SHALL:

- Enforce declared capability boundaries
- Enforce version pinning
- Enforce stamp validation
- Execute mandatory deterministic pre-execution checks
- Operate fail-closed

I.2.3 Non-Bypassable Pre-Execution Enforcement

Before any sandboxed artifact or workflow capsule may execute in a trusted or production context:

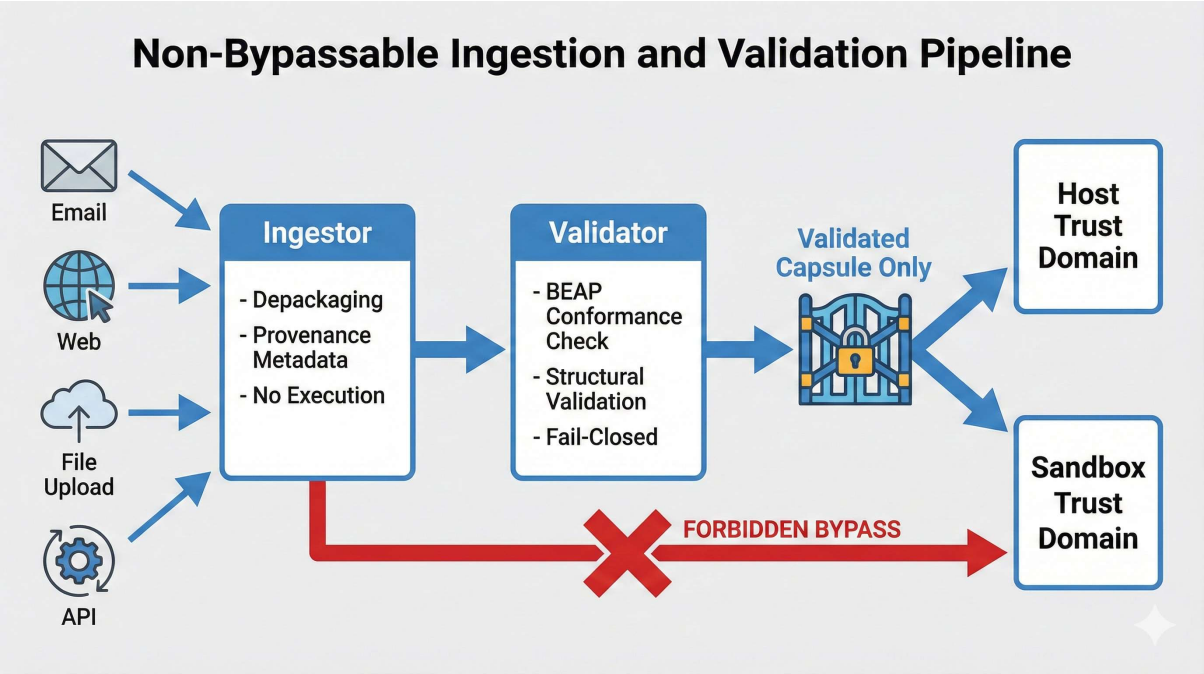
- Mandatory deterministic test routines SHALL be executed
- Stamp integrity SHALL be verified
- Capability declarations SHALL be validated
- Policy conformance SHALL be confirmed

These checks SHALL NOT be deferred, disabled, or bypassed.

Unverified logic SHALL NEVER execute in a trusted domain.

I.2.4 Physical Separation Requirement

Sandbox sub-orchestrators SHALL execute on physically separated hardware.
Virtualization or containerization alone SHALL NOT be considered sufficient isolation.



I.3 Ingestion and Validation Pipeline (Normative)

I.3.1 Ingestor–Validator Separation

The system SHALL implement a mandatory two-stage ingestion pipeline:

- Ingestor
- Validator

These components SHALL be logically separated with a non-bypassable interface.

I.3.2 Ingestor Responsibilities

The Ingestor SHALL:

- Act as exclusive entry point for all external inputs
- Depackage inputs into candidate BEAP™ capsules
- Attach provenance metadata

The Ingestor SHALL NOT:

- Perform full protocol validation
- Render content
- Execute payloads
- Distribute capsules

I.3.3 Validator Responsibilities

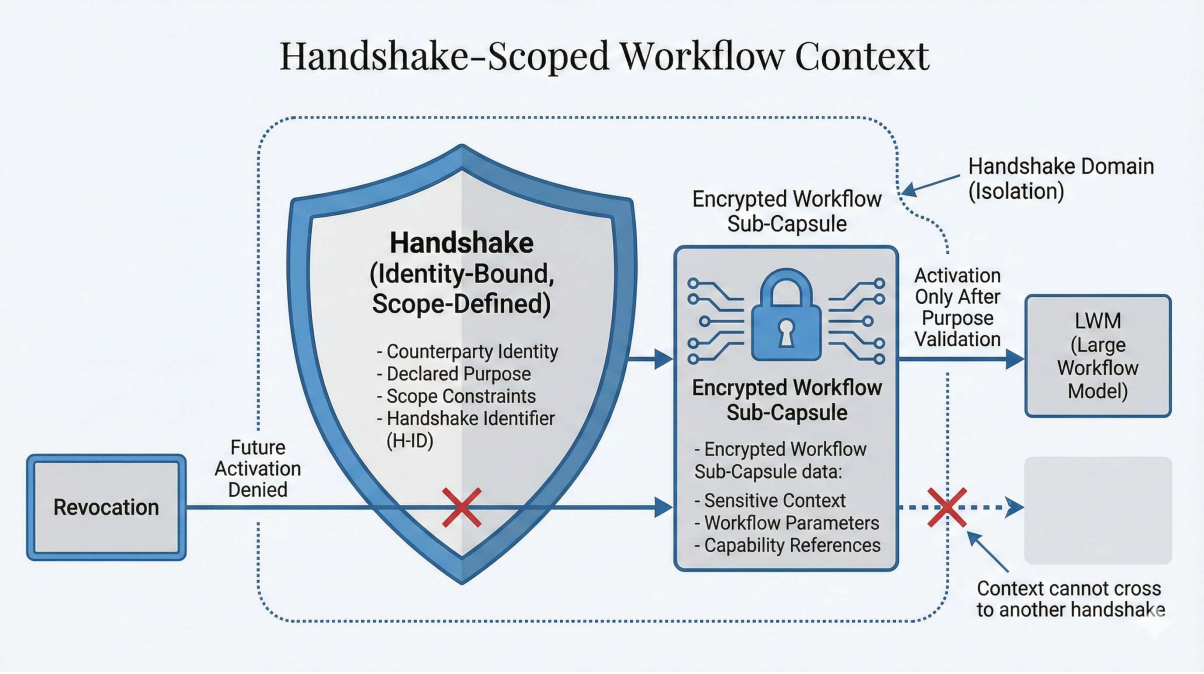
The Validator SHALL:

- Perform strict BEAP™ conformance validation
- Validate structural integrity and required metadata
- Fail closed on any error
- Mark capsules as validated only upon full verification

No component other than the Validator may mark a capsule as validated.

I.3.4 Distribution Gate

Only capsules explicitly marked as validated SHALL be distributed to any trust domain.



I.4 Workflow Capsule Governance

I.4.1 Workflow Capsule Classification

Workflow capsules SHALL be classified as governance artifacts.

They:

- SHALL NOT be directly executable
- SHALL require mediated ingestion
- SHALL require capability validation
- SHALL be subject to handshake scope rules

I.4.2 Handshake Binding of Workflow Context

If workflow capsules are exchanged under a handshake:

- They SHALL be cryptographically bound to the handshake identifier
- They SHALL inherit handshake lifetime constraints
- Revocation SHALL invalidate future activation
- Workflow semantics SHALL NOT be replayable across handshake boundaries

I.4.3 Replay and Idempotency

Replay evaluation SHALL consider:

- Idempotency mode
- Handshake state
- Node execution state
- PoAE™ logs (if available)

Replay SHALL NOT bypass:

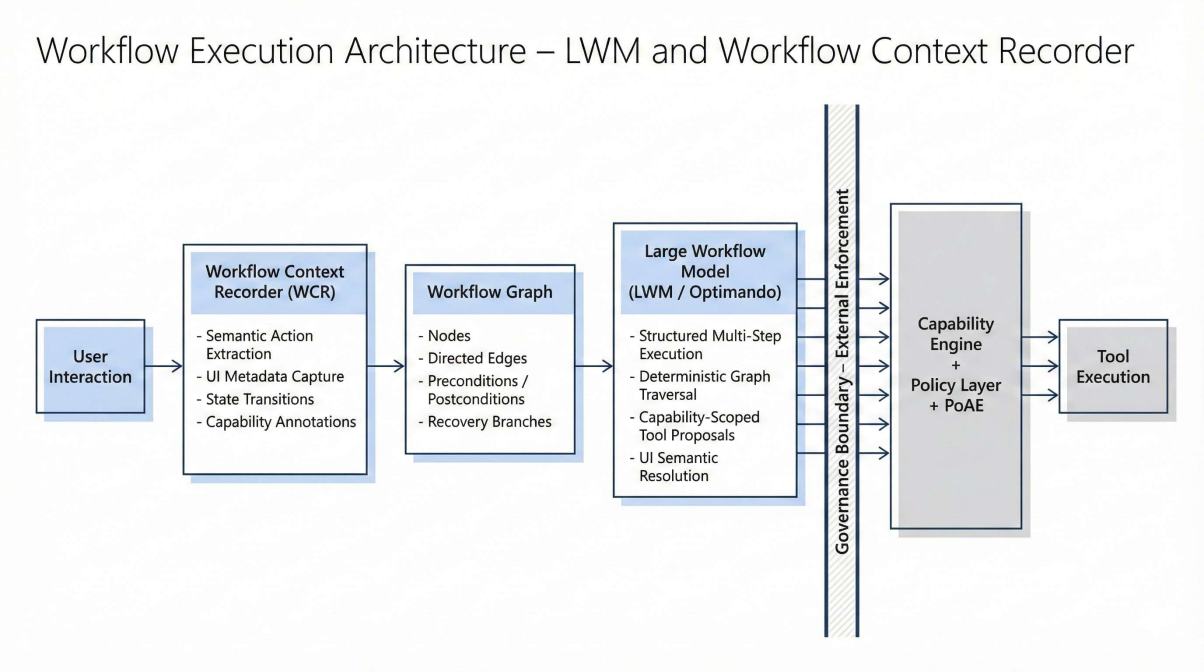
- Capability constraints
- Attestation requirements
- Purpose identifiers
- Consent conditions

I.4.4 Sensitive Workflow Context

Sensitive workflow elements:

- SHALL be stored exclusively within encrypted sub-capsules
- SHALL remain inactive until purpose-bound activation
- SHALL NOT influence deterministic execution logic prior to activation

---



I.5 Large Workflow Model (LWM) Enforcement

I.5.1 Definition

The Large Workflow Model (LWM / Optimando) is a workflow-specialized derivative of a foundation model optimized for:

- Structured multi-step task execution
- Deterministic workflow graph traversal
- Capability-scoped tool invocation
- UI abstraction and semantic resolution
- Recovery-aware reasoning

I.5.2 Authority Constraints

The LWM SHALL NOT constitute an autonomous authority layer.

Execution authority SHALL remain governed by:

- Capability engine
- Policy engine
- PoAE™ enforcement
- Attestation validation

I.6 Workflow Context Recorder (WCR)

I.6.1 Semantic Extraction Requirement

The Workflow Context Recorder SHALL:

- NOT record executable macros
- NOT produce replay scripts
- NOT bind execution to static selectors or pixel coordinates

It SHALL extract:

- Chronological semantic action sequences
- UI metadata (roles, labels, attributes)
- Structural DOM relationships
- Input-output dependencies
- State transitions
- Capability annotations

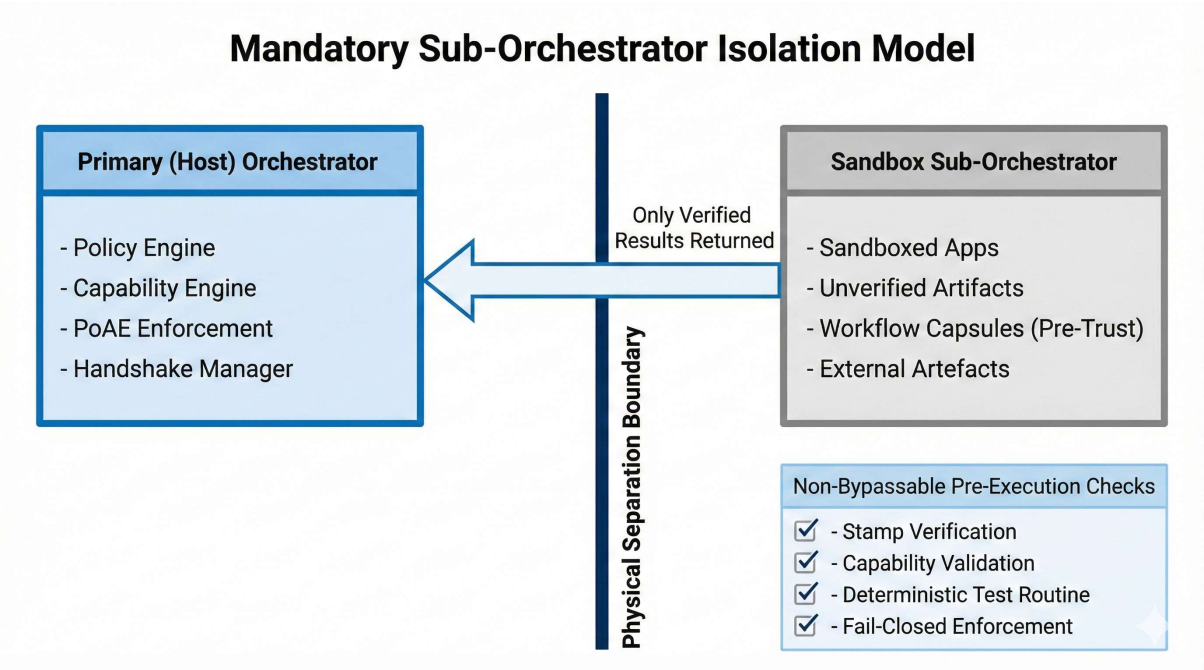
I.6.2 Structured Representation

Recorded workflows SHALL be transformed into workflow graphs containing:

- Semantic nodes
- Directed edges
- Capability annotations
- Optional recovery branches

Workflow capsules SHALL be structured textual artifacts.

They SHALL require mediated interpretation under BEAP™ governance.



I.7 Deterministic LLM-Governed App Ecosystem (Informative)

I.7.1 Sandboxed-by-Default Model

All applications and extensions SHALL begin with a mandatory sandbox label.
Trust escalation SHALL be explicit and never implicit.

I.7.2 Stamped Code Capsules

Artifacts SHALL be distributed as stamped code capsules binding:

- Executable code
- Dependencies
- Declared capabilities
- Identity
- Lineage metadata

Updates SHALL produce new stamped capsules with explicit parent references.

I.7.3 Network-Enforced Swarm Verification

Sandboxed artifacts SHALL undergo mandatory distributed deterministic testing.

- Test bundles SHALL be hash-validated and whitelisted
- Trust progression SHALL follow fail-closed semantics

Marketplace eligibility SHALL require:

- Sufficient valid test artifacts
- Coverage across bundle families
- Participation from higher-trust identities

I.7.4 Deterministic LLM Evaluation

Formal trust decisions SHALL rely on deterministic evaluation:

- Pinned model versions
- Deterministic decoding
- Schema-validated outputs
- Reproducible pipelines

Evaluations SHALL produce hard pass/fail gates.

I.8 High-Assurance Mode

In High-Assurance Mode:

- All executable components SHALL be WRStamped
- All versions SHALL be pinned
- All execution SHALL be identity-bound
- Dynamic uncontrolled runtime code SHALL be excluded

Unverifiable content SHALL be routed off-band.

I.9 Security and Determinism Invariants

The enforcement model SHALL ensure:

- Processing restricted to declared purposes
- No implicit data access
- Full auditability
- Attestation-bound activation
- Strict determinism

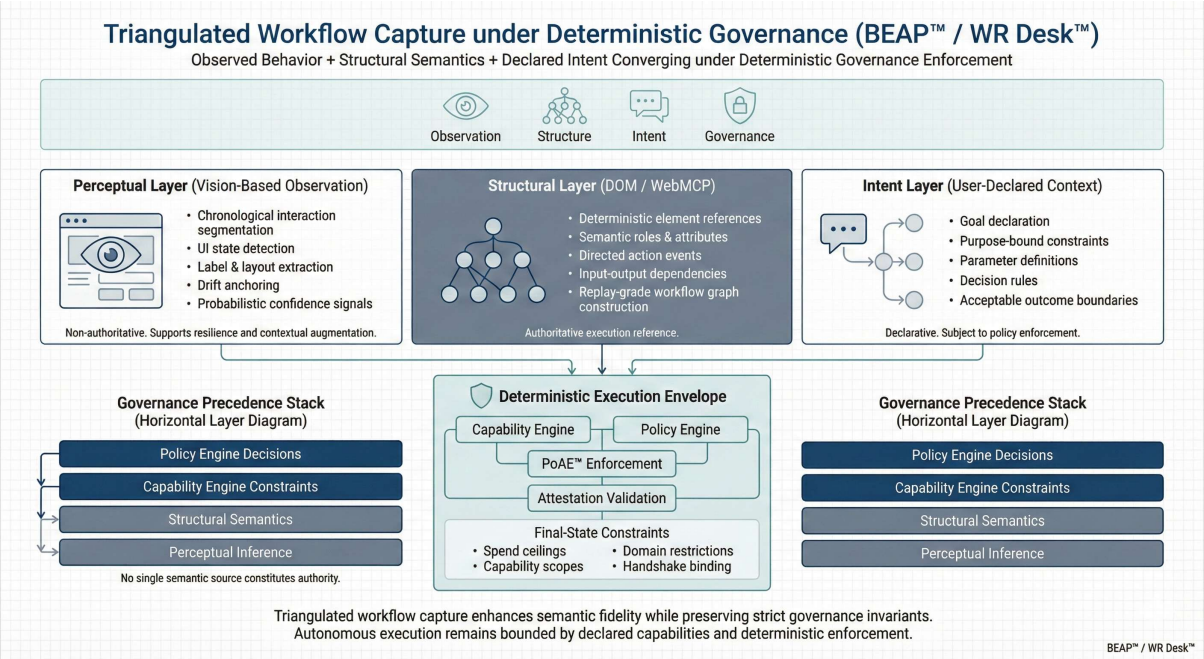
Handshake revocation SHALL immediately affect future activation without altering historical integrity.

I.10 Staged Evolution

This Annex defines the target enforcement architecture.

Partial implementations do not reinterpret this specification.

Conforming systems SHALL evolve toward full compliance without diluting guarantees.



I.11 Triangulated Workflow Capture Model (Informative)

I.11.1 Purpose and Rationale

This section describes the tri-layer workflow capture model employed by WR Desk™ for high-fidelity semantic workflow reconstruction.

It clarifies how observational signals, deterministic structural metadata, and user-declared intent are combined to produce governance-bound workflow capsules.

This section is informative and does not modify normative requirements defined elsewhere in this Annex or in the Canon.

I.11.2 Three-Layer Capture Architecture

Workflow capture MAY integrate three independent but synchronized semantic layers:

1. **Perceptual Layer (Vision-Based Observation)**
2. **Structural Layer (DOM / WebMCP / Deterministic Interface Metadata)**
3. **Intent Layer (User-Declared Semantic Context)**

These layers serve distinct roles and SHALL NOT be conflated.

I.11.3 Perceptual Layer (Vision-Based Observation)

The perceptual layer:(Vision AI, Smartglass, Camera)

- Observes rendered UI states
- Segments chronological interaction sequences
- Detects state transitions
- Extracts visible labels and layout context
- Provides recovery anchoring in case of structural drift

The perceptual layer:

- Is probabilistic in nature
- SHALL NOT constitute execution authority
- SHALL NOT override structural validation
- MAY provide confidence signals for reconciliation

Its primary role is resilience and contextual augmentation.

I.11.4 Structural Layer (Deterministic Interface Semantics)

The structural layer:

- Extracts DOM relationships or equivalent interface metadata
- Resolves semantic roles, identifiers, and attributes
- Records directed action events
- Captures input-output dependencies
- Establishes deterministic execution references

The structural layer:

- Constitutes the authoritative execution reference
- Enables reproducible workflow graph construction
- Supports deterministic replay under governance
- Anchors capability validation

When available, structural signals SHALL take precedence over perceptual hypotheses for execution semantics.

I.11.5 Intent Layer (User-Declared Semantic Context)

The intent layer:

- Captures user-declared goals
- Captures purpose-bound constraints
- Defines parameterization
- Defines decision rules
- Defines acceptable outcomes

The intent layer:

- SHALL NOT directly execute logic
- SHALL NOT expand capability scopes

- SHALL remain subject to PoAE™ enforcement
- SHALL bind workflows to declared purposes

Intent declarations augment semantic meaning but do not override deterministic enforcement layers.

I.11.6 Synchronization and Alignment

All three layers MAY be time-aligned via chronological indexing.

During workflow graph construction:

- Structural data defines executable nodes
- Intent data defines semantic purpose bindings
- Perceptual data assists drift handling and anomaly detection

Conflicts SHALL be resolved according to governance precedence:

1. Policy engine decisions
2. Capability engine constraints
3. Structural layer semantics
4. Perceptual inferences

Intent descriptions SHALL remain declarative and non-authoritative.

I.11.7 Deterministic Envelope for Autonomous Execution

When combined with:

- Capability enforcement
- Policy validation
- PoAE™ constraints
- Attestation-bound execution contexts

The tri-layer capture model enables a deterministic execution envelope.

Autonomous execution SHALL remain bounded by:

- Explicit capability declarations
- Final-state constraints
- Spend or action ceilings
- Domain or scope restrictions
- Handshake bindings (if applicable)

No workflow SHALL exceed declared and validated enforcement constraints, regardless of inferred intent.

I.11.8 Non-Executable Recording Principle

The Workflow Context Recorder (WCR):

- Does not produce executable macros
- Does not bind to pixel coordinates
- Does not generate uncontrolled replay scripts

Instead, it produces governance-bound workflow graphs requiring mediated interpretation under BEAP™.

I.11.9 Assurance Implications

In High-Assurance Mode:

- Tri-layer capture MAY be used to increase semantic fidelity
- Deterministic structural validation remains mandatory
- Perceptual inference SHALL NOT relax enforcement constraints
- All resulting workflow capsules remain subject to handshake binding and attestation requirements

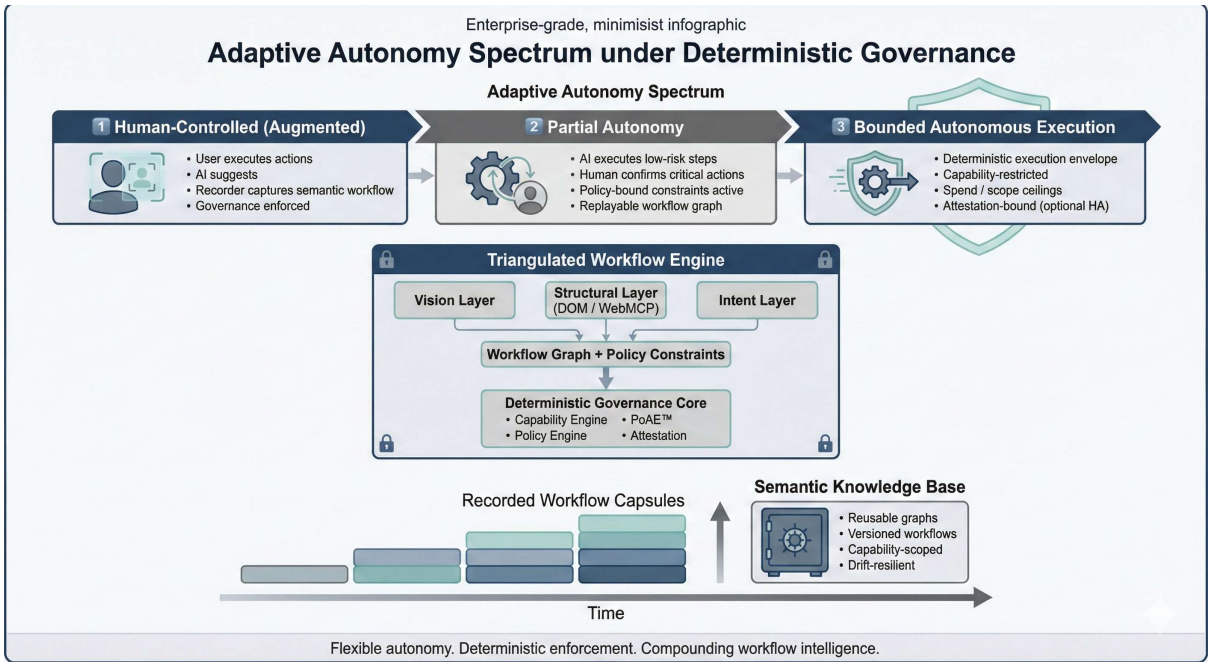
Tri-layer capture enhances robustness but does not modify governance invariants.

I.11.10 Design Principle

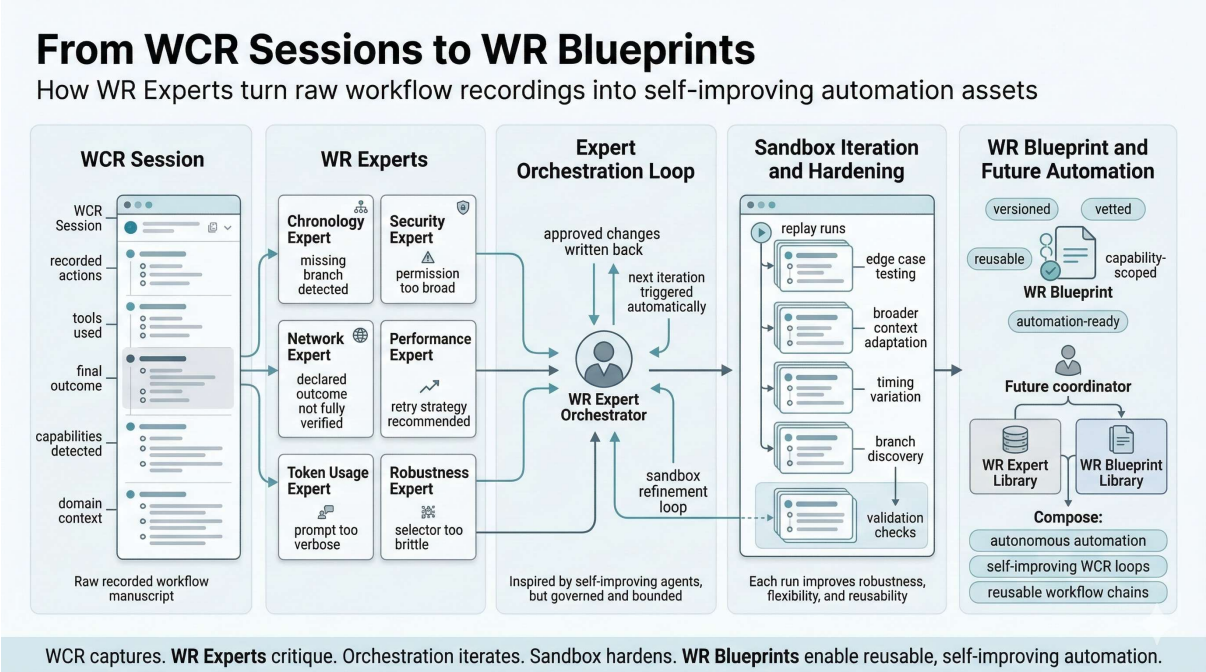
The tri-layer model embodies the following principle:

Observed behavior, structural reference, and declared intent SHALL converge under deterministic governance enforcement.

No single semantic source SHALL constitute authority.



The long-term vision is a flexible autonomy spectrum built on a constant governance core. Users could operate in fully human-controlled (augmented), partially autonomous, or strictly bounded autonomous modes—without changing the underlying enforcement architecture. Over time, recorded workflows would evolve into structured, policy-bound knowledge assets, forming a reusable and drift-resilient execution layer. Autonomy would scale progressively, but always remain constrained by declared capabilities, explicit purpose, and verifiable governance controls.



WCR captures. **WR Experts** critique. Orchestration iterates. Sandbox hardens. **WR Blueprints** enable reusable, self-improving automation.

From WCR Sessions to WR Blueprints: Expert-Orchestrated Workflow Hardening

A Workflow Context Recorder should do more than capture what happened during a session. Its real value begins when a recorded workflow can be refined into a reusable, trustworthy artifact. That is where **WR Blueprints** and **WR Experts** become important.

A **WR Blueprint** is the vetted form of a recorded workflow. It is not just a log of clicks, inputs, and outputs, but a structured and versioned representation of how a task was completed, which capabilities were required, what outcome was achieved, and which constraints, risks, and edge cases were identified. A raw WCR session becomes operationally useful only once it has been hardened into a WR Blueprint.

For routing and reuse, title and description alone are not enough. A stronger model should also consider the tools actually used, the final outcome reached, the capability profile, the domain context, and the observed execution pattern. These signals are more reliable for machine routing than human-written labels alone and make it easier to find relevant workflows later.

This is where **WR Experts** come in. WR Experts are specialist evaluators that review a WCR session from different angles and provide concise, structured feedback. One expert may focus on chronological accuracy and whether the sequence of actions is complete. Another may analyze network behavior to verify that the workflow behaves as declared. A security expert may identify risks, overbroad permissions, or unsafe side effects. Other experts may focus on performance, token usage, robustness, recoverability, capability minimization, or broader execution quality.

The key idea is that WR Experts do not have to operate in isolation. They can themselves be **orchestrated**. Their findings can be merged by a coordinator that writes approved changes directly back into the **WCR manuscript** and automatically triggers the next iteration loop. This creates a bounded form of recursive improvement inspired by self-improving agent systems: one expert identifies a weakness, another proposes a stabilization, the coordinator

incorporates the change, and the workflow is replayed again under sandbox conditions. The result is not uncontrolled self-modification, but structured iterative hardening.

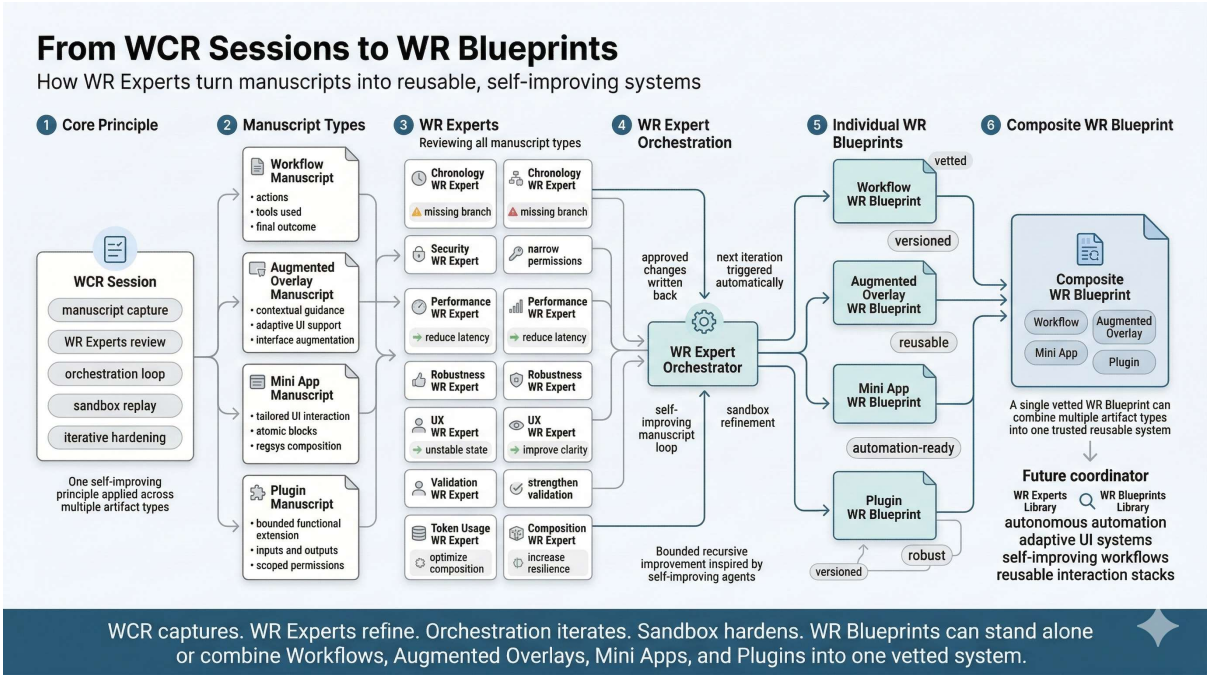
This makes the recorder more than a passive capture tool. During recording, WR Experts can surface live feedback in a grid or panel so the human operator sees quality signals while the workflow is still being authored. The system can point out missing branches, brittle selectors, weak outcome definitions, redundant steps, or unnecessary permissions. Low-risk improvements can be applied immediately to the sandbox version, while higher-impact changes remain suggestions until validated.

After recording, the workflow should be run several times in a sandbox. Each replay can test edge cases, timing variations, partial failures, or broader use contexts. The purpose is to make the workflow more flexible, robust, and reusable. With every iteration, the WCR manuscript can be refined, weaknesses can be reduced, and the workflow can move closer to the quality expected of a WR Blueprint.

A coordinator layer becomes especially powerful once a larger ecosystem exists. Over time, the coordinator should be able to search the **WR Expert library** and the **WR Blueprint library** to assemble highly autonomous, self-improving automation loops. In that model, existing WR Experts can be reused as evaluators, while proven WR Blueprints can be chained together as building blocks for more advanced workflows. The system can then not only record and improve workflows, but also compose and refine new ones using previously validated assets.

That creates a strong progression model. WCR captures the session. WR Experts analyze and improve it. The coordinator orchestrates iterations and feeds accepted changes back into the manuscript. Sandbox replays test for stability and edge cases. Once the workflow has matured and passed the required checks, it is promoted to a WR Blueprint.

In this form, WR Blueprints and WR Experts are not just naming additions. Together they define a scalable path from raw workflow capture to expert-guided, self-improving, reusable automation.



From Manuscripts to WR Blueprints: Self-Improving Workflows, Augmented Overlays, Mini Apps, and Plugins

The same principles used in a self-improving **WCR session** can be applied not only to workflows, but also to **Augmented Overlays**, **Mini Apps**, and even **plugins**. The WCR model should therefore be understood as a general method for creating, refining, and hardening reusable operational artifacts. In all of these cases, the process begins with a manuscript and matures through iteration into a vetted **WR Blueprint**.

A workflow may begin as a captured sequence of actions and outcomes. An Augmented Overlay may begin as a manuscript describing contextual interface guidance. A Mini App may begin as a structured description of a tailored UI interaction assembled from trusted atomic blocks. A plugin may begin as a bounded functional extension with clearly defined inputs, outputs, permissions, and responsibilities. In every case, the first version is only a draft. It defines intent and structure, but it is not yet a hardened reusable asset.

Just like in a WCR session, **WR Experts** can evaluate these manuscripts from multiple perspectives while they are being created and refined. Depending on the artifact type, WR Experts may assess chronology, security, robustness, logic flow, state handling, UX quality, token usage, performance, composition quality, validation coverage, or contextual relevance. Their role is to provide concise, structured feedback that improves the manuscript without weakening control or clarity.

These **WR Experts** can also be orchestrated. Their findings can be merged by a coordinator that writes approved changes directly back into the manuscript and automatically triggers the next iteration loop. This creates a bounded self-improvement process inspired by autonomous agents: weaknesses are detected, refinements are proposed, low-risk changes are applied, and the result is tested again. In that way, the manuscript improves continuously during and after authoring.

The same replay and hardening model from WCR sessions applies across all artifact types. A workflow can be replayed across edge cases and broader task contexts. An Augmented Overlay can be tested across different layouts, interface states, and user situations. A Mini App can be exercised across different inputs, flows, and UI conditions. A plugin can be validated across dependency, permission, and execution scenarios. Each accepted improvement creates a stronger next version.

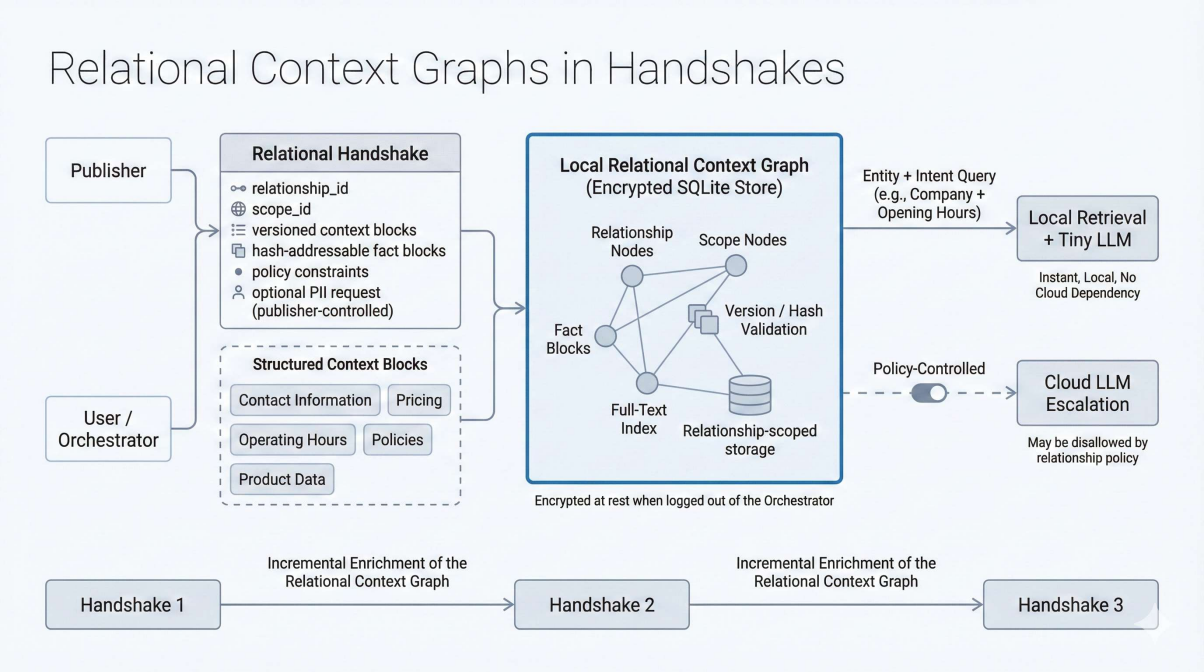
This is where **WR Blueprints** become central. The same principles of the WCR session can be applied to create **WR Blueprints** for workflows, Augmented Overlays, Mini Apps, and plugins. A raw manuscript becomes a candidate, **WR Experts** refine it, sandbox iterations harden it, and once it is sufficiently mature, it is promoted into a vetted **WR Blueprint**.

A particularly important extension is that a **WR Blueprint** does not have to remain limited to a single artifact type. A later-stage **WR Blueprint** can also combine **workflows, Augmented Overlays, Mini Apps, and plugins into one single vetted WR Blueprint**. This means the final artifact is not just an isolated reusable component, but a higher-order operational package. A workflow can include embedded Mini Apps for tailored interaction, Augmented Overlays for contextual guidance, and plugins for bounded functional extension, all governed and validated together as one vetted unit.

This creates a much more powerful model for reuse. Instead of storing only separate assets, the system can also maintain composite **WR Blueprints** that unify multiple layers of behavior and interaction. Over time, libraries of **WR Blueprints** and **WR Experts** can

support increasingly advanced compositions, where proven components are not only reused individually but assembled into larger self-improving systems.

The lifecycle remains consistent across all domains: capture or author a manuscript, let **WR Experts** evaluate it, orchestrate improvements, replay and harden it in sandbox, and promote the mature result into a **WR Blueprint**. From there, vetted **WR Blueprints** can also be combined into more comprehensive **WR Blueprints** that integrate workflows, Augmented Overlays, Mini Apps, and plugins in a single reusable and trusted artifact. That is what turns the WCR approach from a recording method into a scalable architecture for self-improving systems.



Local Persistence and Incremental Enrichment – Relational Context Graph

As part of the relational handshake, exchanged context is not treated as transient prompt material but as durable relationship state. Each handshake may carry one or more semantically defined *context blocks* (fact blocks and/or structured semantic text) that are explicitly bound to a `relationship_id` and optionally to a `scope_id` (e.g., product, location, contract, session). These blocks are versioned and hash-addressable, enabling deterministic identification, deduplication, and update detection. A block is typically *created once* (initial publication), and subsequent handshakes or messages may either reference the existing block by `(block_id, hash)` or enrich the relationship by attaching incremental relational context data (relationship-specific facts) and/or general context data (publisher-wide facts). This yields an append/enrich model rather than repeated re-transmission of the full context payload.

Local Encrypted Storage Model

Upon receipt, the orchestrator persists the handshake-derived context blocks into a local SQLite knowledge store to make them immediately searchable and LLM-native at low latency. Storage is organized around relationship scoping (e.g., indexing by

`relationship_id`, `publisher_id`, `topic/type`, `valid_until`) to enable fast entity+intent retrieval across the accumulated archive of BEAP capsules and handshake context. The SQLite store is encrypted at rest when the orchestrator is logged out: on logout, encryption is enforced by closing the database, removing keys from memory, and rendering the store unreadable until the next authenticated login session. When logged in, the orchestrator can query the local store directly, enabling instant retrieval and local processing without cloud dependency.

Policy-Governed Escalation Path

For increased reasoning capacity or broader context windows, the orchestrator may provide a user-controlled escalation toggle to route selected requests to cloud-based LLMs. This escalation is not implicit: it is explicitly gated by the active relationship policy negotiated in the handshake (and by any applicable local policy). If escalation is disallowed by policy, the orchestrator must remain local-only and restrict processing to locally available context and models. If allowed, the orchestrator can selectively transmit only the minimal required snippets derived from the local context graph, preserving the relationship-scoped constraints while enabling higher-tier inference when needed.

ScamWatchdog™ is a specialized language model developed for secure workflow support, scam and fraud detection, email classification, and contextual risk assessment. It is designed to identify, analyze, and assess digital fraud scenarios across user, application, and system contexts.

The model supports both manually triggered scan sessions and an adaptive baseline protection layer for continuous fraud prevention. When a manual scan is initiated, the local

orchestrator captures relevant screenshots from the workstation and, where available, extracts structured signals from active web content, including DOM-based indicators. Depending on the operational context, the system may also collect and evaluate additional telemetry such as ingress and egress activity, process behavior, session metadata, and other host- or network-level signals.

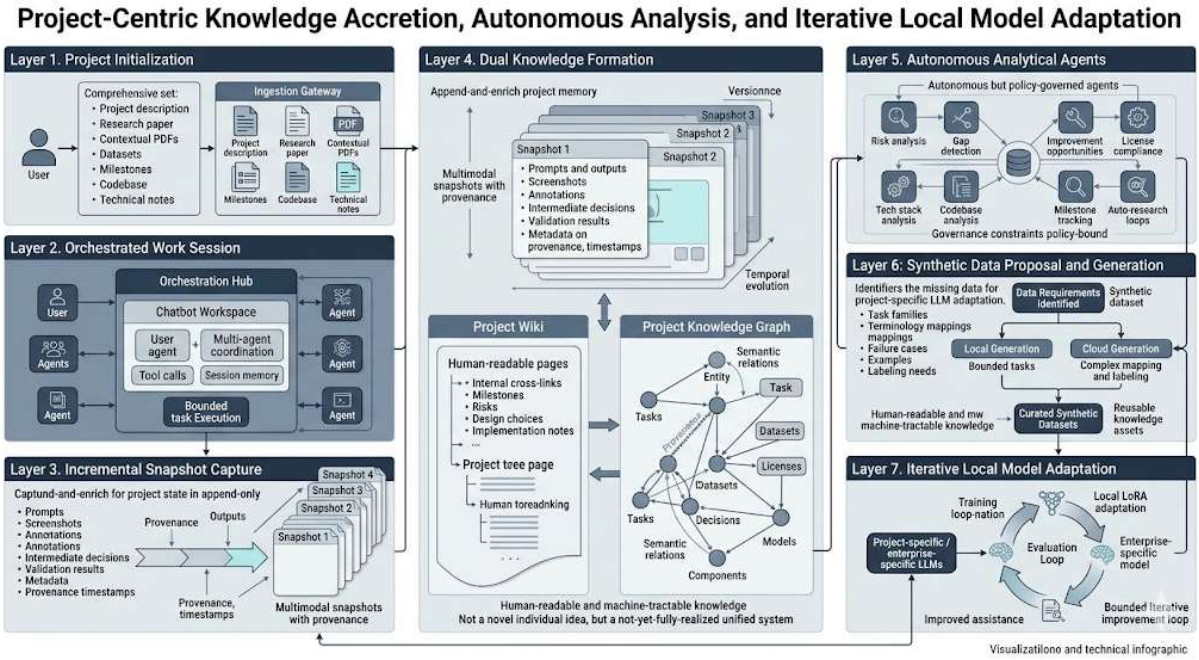
In addition to user-initiated scans, ScamWatchdog™ can be supported by one or more autonomous AI agents operating in scheduled and event-driven modes. At defined intervals, the system can generate lightweight system snapshots for periodic evaluation. More intensive analysis can also be triggered dynamically when elevated-risk contexts are detected, such as email activity, private or business messaging, social media usage, online shopping, marketplace interactions, or digital goods transactions. This allows the system to increase scrutiny only when relevant activity is observed, preserving system resources while maintaining effective fraud monitoring coverage.

Autonomous agents SHALL operate with read-only permissions by default. Subject to the active local configuration and policy settings, an agent MAY autonomously gather information relevant to the specific matter under analysis from locally available sources and, where explicitly enabled, from internet-based sources. Any action requiring autonomous write-permissions SHALL remain cryptographically gated, policy-bound, and deterministically enforced. This ensures that adaptive information gathering is possible while all state-changing operations remain controlled, auditable, and deterministic where it matters most.

ScamWatchdog™ processes multimodal and contextual inputs, including screenshots, browser-derived signals, system telemetry, and optional supplementary user inputs such as text, images, or audio files. Based on these inputs, it produces a consolidated risk assessment that identifies scam indicators, behavioral anomalies, and other fraud-relevant patterns. The output can be presented as a structured risk report with prioritized indicators and visualized risk information to support user awareness and decision-making.

The model is particularly effective when deployed with OS-level visibility, where it can correlate behavioral, visual, and technical indicators across applications and sessions. At the same time, orchestration-layer analysis can complement this approach by evaluating available signals independently of full host-level inspection. The underlying model is optimized for fraud-related anomaly detection and trained on scenarios including scam detection, phishing detection, digital fraud prevention, and abnormal interaction patterns across user and system activity.

ScamWatchdog™ is one domain-specific model within the broader WRGuard™ architecture. WRGuard™ acts as the intelligent smart-routing, orchestration, and agent framework that coordinates which model, tool, or application should be used for a given task. Rather than relying on one large general-purpose model, WRGuard™ is designed as a modular ecosystem of highly specialized smaller models, each focused on a distinct operational domain



I.12 Automated Project-WIKI for synthetic trainings-data generation, iterative local llm finetuning and llm-based deep analysis (Informative)

I.12.1 Purpose and Scope

This section describes an extended project-centric operating mode in which the orchestrator is used not only for bounded workflow execution, but also for continuous project-state accumulation, structured knowledge formation, and iterative model adaptation. In this mode, a user may initialize a project by providing an explicit project description together with relevant artefacts, including research papers, contextual PDFs, datasets, milestones, technical documentation, code repositories, and other domain-specific materials. The objective is to transform these heterogeneous inputs, together with subsequent interaction traces, into a durable and progressively enriched project memory that supports both human understanding and machine-guided analysis.

I.12.2 Incremental Snapshot Formation

As work on a project proceeds through orchestrated chatbot interaction, the system may generate incremental snapshots of project state. A snapshot may include user prompts, agent outputs, retrieved context, structured annotations, intermediate decisions, validation results, screenshots, and other multimodal evidence associated with the session. Rather than treating these materials as transient conversational residue, the system may persist them as versioned project-state artefacts with explicit provenance, temporal ordering, and scope binding. This enables an append-and-enrich model in which project knowledge is accumulated incrementally, revised without loss of lineage, and queried as a temporally evolving body of evidence rather than as a static document.

I.12.3 Project Wiki and Knowledge Graph Synthesis

From the accumulated project-state artefacts, the system may derive two complementary representational layers: a project wiki and a project knowledge graph. The wiki serves as a human-readable, continuously updated synthesis layer that organizes project intent, artefacts, milestones, assumptions, risks, design choices, implementation notes, and outcomes into internally linked narrative pages. The knowledge graph serves as the corresponding machine-oriented substrate, representing entities, claims, tasks, milestones,

dependencies, decisions, risks, software components, datasets, licenses, and related project objects as typed nodes connected by semantically meaningful relations. In this formulation, the wiki is not the primary source of truth, but a rendered and navigable projection over a provenance-preserving graph of project knowledge. Internal links may therefore be established and revised automatically as new evidence is added, relations are inferred, or obsolete assertions are superseded.

I.12.4 Autonomous Multi-Agent Analytical Sessions

A single allocated session may host multiple bounded analytical tasks operating over the same project context. These tasks may include, but are not limited to, risk identification, gap analysis, improvement discovery, milestone tracking, license assessment, technology-stack evaluation, architecture review, and codebase analysis. Specialized agents may inspect the same project state from different perspectives and contribute structured findings back into the shared project memory. Where appropriate, a coordinator layer may reconcile overlapping observations, detect contradictions, merge compatible updates, and schedule further targeted analysis. Auto-research style loops may also be applied to specific subproblems, enabling deeper iterative investigation of unresolved technical, scientific, or organizational questions. Such operation does not imply unconstrained autonomy: analytical agents remain subordinate to the governing orchestration layer, and any state-changing action remains policy-bound, reviewable, and subject to the applicable enforcement model.

I.12.5 Identification of Fine-Tuning Data Requirements

Beyond documentation and analysis, the system may inspect the project as an adaptation target for local language models. Multi-agent review may be used to determine which knowledge components, task formats, failure cases, taxonomies, terminology mappings, exemplars, and decision traces would be most valuable for model specialization. In this setting, the system does not merely retrieve existing materials; it also identifies coverage deficits and proposes what additional data should exist in order to support domain adaptation. Candidate fine-tuning data may therefore be specified in terms of desired task families, target behaviors, evaluation criteria, domain constraints, safety requirements, and representational granularity.

I.12.6 Synthetic Dataset Generation and Curation

Once proposed data requirements are accepted, the system may generate synthetic datasets derived from the project's documented knowledge. Generation may occur locally when the available reasoning capacity is sufficient, particularly for bounded transformation, templating, augmentation, or low-complexity labeling tasks. More complex mapping, decomposition, alignment, or semantic labeling tasks may be delegated to cloud-based models when permitted by the active policy and scope constraints. Synthetic data generation is not treated as an opaque side process; generated samples may be linked back to their originating project artefacts, assumptions, and target objectives, thereby preserving traceability and enabling subsequent review. Curated datasets may then be versioned as first-class project assets and incorporated into the project knowledge graph as reusable knowledge objects.

I.12.7 Iterative Local Model Adaptation

Curated synthetic datasets may be used for iterative local adaptation of project-specific or enterprise-specific models, including parameter-efficient fine-tuning methods such as LoRA. In this loop, a locally adapted model is evaluated against project-relevant tasks, failure modes, and domain constraints, and the resulting observations may themselves be

reintegrated into the wiki and knowledge graph as new evidence. The overall process therefore forms a recursive but bounded improvement cycle: project work produces structured knowledge; structured knowledge yields candidate training data; candidate training data enables local specialization; specialized models generate more precise project assistance; and their performance informs subsequent project refinement. The intended outcome is not merely a better chatbot, but a project-aligned or organization-aligned model ecosystem capable of applying tailored knowledge with higher precision and lower contextual ambiguity.

I.12.8 Domain Generality

The proposed framework is domain-agnostic. A project may correspond to scientific research, engineering, software development, internal workplace optimization, operational governance, or other structured knowledge-intensive activities. Across these domains, the common objective is to document evolving knowledge in a machine-tractable form and to use that documentation both as an operational memory layer and as a basis for model adaptation. In this sense, the project wiki and the knowledge graph function simultaneously as documentation infrastructure, analytical substrate, and data-generation scaffold.

I.12.9 Positioning of the Contribution

The conceptual elements described here are not individually novel. Project documentation systems, knowledge graphs, autonomous analysis agents, synthetic data generation, retrieval-augmented workflows, and local fine-tuning methods have all been explored independently in prior work and in existing systems. The contribution of the present formulation lies instead in their integration into a single continuous project loop in which documentation, graph formation, analytical augmentation, synthetic dataset curation, and local model adaptation are treated as mutually reinforcing stages of one operational architecture. While the constituent ideas are established, their comprehensive realization as a unified, end-to-end, project-centric system remains insufficiently implemented in practice.

I.13 Deployment Scope of BEAP™ (Normative)

I.13.1 Current Deployment Scope

BEAP™ is currently specified, implemented, and validated exclusively for deployment within:

- Optirando™ / WR Desk™ environments
- Silicon Cargo environments

Within this scope, all conformance guarantees of the Canon and this Annex apply in full.

BEAP™ SHALL NOT be represented, marketed, or relied upon as a general-purpose third-party interoperability standard at this stage. Conformance claims by implementations outside the scope above are undefined.

I.13.2 Future Third-Party Opening (Informative)

Opening BEAP™ to third-party implementers is a long-term objective, not a present property of the protocol. Such an opening would require, at minimum:

- a published conformance test suite,
- capsule and gateway certification procedures,
- a governance process for protocol evolution.

Nothing in this Annex implies a timeline or commitment for this opening. Until such a program exists, references to third-party use SHALL be framed as outlook only.

I.14 WR Graph™ Adapter Activation Semantics (Normative)

I.14.1 Adapter Bindings as Graph Content

WR Graph™ nodes and subgraphs MAY carry bindings to fine-tuned, LoRA-class domain adapters as first-class context. An adapter binding associates a context region (device class, workflow stage, topic, publisher relationship) with the parametric competence required to reason within it.

I.14.2 The WR Graph™ as Activation Trigger Layer

The WR Graph™ SHALL act as the authoritative trigger layer for adapter lifecycle operations, namely:

- **Activation** — loading an adapter when its bound context becomes active
- **Hot-swap** — replacing the active adapter when context resolution changes
- **Combination** — composing multiple adapters where a context legitimately spans several bound domains
- **Deactivation** — unloading adapters whose bound context is no longer active

Lifecycle operations SHALL be triggered exclusively by deterministic context-resolution events, including:

- WR Code® scan resolution
- workflow state transitions within an active recorded workflow
- topic classification of the active session against the graph
- handshake scope activation (see I.15)

I.14.3 Determinism and Authority Constraints

- Adapter selection SHALL be deterministic: identical context resolution against identical graph state SHALL yield the identical adapter set.
- Adapter activation SHALL NOT expand capability scopes. Capabilities remain exclusively governed by the capability engine, policy engine, and PoAE™ enforcement.
- Every adapter lifecycle event SHALL be logged with context reference, graph version, and adapter identity, and SHALL be auditable.
- Adapters and their weights SHALL remain within the user's trust domain (local Host AI); the graph selects, the host loads.

I.14.4 Multidimensional Graph Structure (Informative)

The WR Graph™ is not a conventional knowledge graph. It comprises multiple interconnected layers over one context space:

1. **Declarative layer** — the structured knowledge graph itself (entities, relations, constraints, decisions)
2. **Episodic layer** — long-term interaction memory bound to graph regions
3. **Parametric layer** — adapter bindings carrying domain competence (I.14.1)
4. **Procedural layer** — embedded Workflow Context Recorder automations (I.6, I.17)
5. **Organizational layer** — background sorting, prioritization, and self-optimization of the above, driven by usage, corrections, and authorized decisions

Cross-layer edges enable a single context resolution (e.g. one scan) to select all layers simultaneously: what is known, what has happened, which competence applies, and what can be executed.

I.14.5 Routing Semantics and the Determinism Boundary (Normative)

The WR Graph™'s routing function is **preparatory, never executive**. It selects and proposes — knowledge regions, adapters, and candidate automation paths — but executes nothing. The determinism guarantee of this specification attaches to the **execution artifact**, not to the routing:

- A finalized WCR recording, including its tool invocations, SHALL be deterministic: identical recording version plus identical declared inputs SHALL yield the identical action sequence.
- Consequential steps SHALL be gated by user consent at the applicable PoAE™ tier **regardless of how the request was routed**. No routing outcome can substitute for, weaken, or pre-satisfy an authorization requirement.
- Routing state SHALL be derived exclusively from local, user-authorized signals (usage, ratings, corrections, authorized decisions). External signals — including publisher feedback, incoming capsules, or gateway traffic — SHALL NOT influence routing state.

Consequently, evolution of the routing layer over time affects only which validated options are surfaced, never what an option does when executed or whether it may execute.

I.14.6 Graph Admission Control (Normative)

LoRA-class adapters and WCR recordings SHALL NOT be admitted into the WR Graph™ automatically. Admission requires explicit user consent, consistent with the principle that trust escalation is explicit and never implicit (I.7.1). Until admitted, such artifacts remain in their respective libraries or in the sandbox trust domain and are not available to routing.

I.14.7 Inspectability and Snapshots (Normative)

The WR Graph™ SHALL be visually inspectable by the user. The system SHALL support taking an immutable **snapshot** of the graph at any point in time, so that its state remains reviewable even as the graph evolves.

Full graph versioning MAY be implemented; in **High-Assurance Mode** (I.8), graph state relevant to routing and automation admission SHALL be versioned, so that any past routing or admission decision can be reconstructed against the graph state under which it was made.

I.14.8 Graceful Degradation and Domain Boundedness (Normative)

Automations are domain-bound execution logic, not a universal interaction mode. If context resolution finds no matching domain region — including the empty-graph case — the system SHALL degrade gracefully:

- no domain adapter is loaded; the normally selected base model answers,
- no automation path is offered,
- no routing is fabricated from insufficient context.

Absence of domain expertise SHALL be surfaced as such, not masked by a low-confidence match.

I.15 Permission-Bearing Asymmetric Handshake and the BEAP™ Gateway

I.15.1 Extension of Handshake Semantics (Normative)

In addition to counterparty identity, declared purpose, scope constraints, and the handshake identifier (H-ID), a handshake MAY carry **automation permission grants**. A permission grant SHALL specify:

- the permitted automation classes or named workflow capsules
- per-step capability ceilings (spend, scope, domain restrictions)
- the PoAE™ tier mapping applicable to each consequential step
- validity period and revocation conditions

Permission grants SHALL be recorded in the relational context graph on the user side **and** in the publisher's backend on the gateway side. Execution SHALL require agreement of both records; any mismatch SHALL fail closed.

Revocation of the handshake SHALL immediately invalidate all future activations under its permission grants, consistent with I.4.2.

I.15.2 BEAP™ Gateway (Normative Definition)

A **BEAP™ Gateway** is a server-adapted variant of the orchestrator, embeddable into a website and its backend. The Gateway:

- SHALL issue and accept handshake requests on behalf of the publisher
- SHALL connect to the publisher's database exclusively through a bounded, declared connector
- SHALL publish WCR-recorded automations as validated workflow capsules
- MAY maintain a catalog of such capsules covering the publisher's recurring support and configuration topics (e.g. its FAQ corpus)
- SHALL be operable only by publishers that have completed DNS-based publisher verification — the same verification already required for the publication of public BEAP™ capsules

The Gateway inherits all orchestrator-side enforcement obligations; it does not introduce a weaker conformance class.

I.15.3 Registration-Coupled Handshake (Informative)

In the ordinary registration flow of a website, the user MAY additionally conclude an asymmetric handshake with the publisher through the site's BEAP™ Gateway. Registration and handshake remain separable acts: the handshake adds a governed automation relationship on top of the conventional account relationship. The publisher thereby gains a realtime, structured channel to the user's orchestrator instead of static or after-the-fact data exchange.

I.15.4 Asymmetric Channel Model (Normative)

The relationship established under I.15.1–I.15.3 is deliberately asymmetric:

Forward channel (publisher → user) MAY carry:

- workflow capsules (WCR-recorded automations)
- context blocks (per the Relational Context Graph model)
- status and versioning information

Backchannel (user → publisher) SHALL be restricted to:

- PoAE™ authorization records

- explicitly declared data items covered by the permission grant (e.g. address data)
- status and outcome messages
- user feedback artifacts (screenshot plus textual description), subject to text-purity and attachment-handling rules

The backchannel SHALL NOT carry arbitrary content, telemetry, capsules, or undeclared data classes. This restriction SHALL be enforced fail-closed at the user's orchestrator. The asymmetry is a privacy and containment property, not an implementation convenience.

I.15.5 Execution Model and Publisher-Sanctioned Automation (Normative)

Automations delivered over the forward channel are recorded by the publisher against the publisher's own surfaces (website, dashboard, configuration console) and executed **in the user's environment** against those surfaces.

The decisive governance point: website operators generally do not permit automation on their surfaces. Under a permission-bearing handshake, the publisher **explicitly sanctions** automation on its own surfaces, and because publisher and surface evolve under one governance, recordings and interface remain aligned (cf. §2.3.3 of the Examples paper).

The following invariants remain unmodified:

- Incoming workflow capsules SHALL traverse the full Ingestor–Validator pipeline (I.3) and remain sandboxed until validated.
- Capsules SHALL be cryptographically bound to the H-ID and inherit handshake lifetime constraints (I.4.2).
- Recorded automations run to the authorization boundary and no further; every consequential step is gated by its PoAE™ tier as mapped in the permission grant.
- The publisher's Gateway is connected to the publisher's own host over an internal handshake, subject to the same enforcement model.

I.15.6 Capsule Catalogs and Graph-Routed Dispatch (Informative)

A publisher will typically author not one automation but a catalog — for example, on the order of a hundred WCR recordings covering the topics of its FAQ, its onboarding flows, and its recurring configuration tasks. Dispatch of a catalog entry to a user is triggered in one of two ways:

1. **Question-routed dispatch.** The user submits a support request in natural language. The WR Graph™'s organizational layer performs the routing: it matches the question against the catalog's semantic annotations so that the correct question triggers the correct recording. The Gateway's server-side orchestrator then transmits exactly that capsule to the user over the P2P connection.
2. **Explicit dispatch.** The user triggers a specific automation directly via a button or link on the publisher's website; the corresponding capsule is transmitted without routing.

In both cases the user's orchestrator receives the capsule through the standard ingestion and validation pipeline, executes it locally under the handshake's permission grants, and the user **rates the outcome** (thumbs up / thumbs down). The rating closes the loop of I.15.7 and, on the publisher side, refines the routing quality of the catalog.

I.15.7 Iterative Support and Feedback Loop (Informative)

Where an automation does not resolve the user's problem, the low-latency P2P relationship enables an iterative loop:

1. Automation runs and fails or terminates incompletely.

2. The user returns a feedback artifact over the backchannel: screenshot plus description ("this did not work").
3. The publisher (or its support tooling) derives a tailored workflow for exactly this state and ships it over the forward channel.
4. The loop iterates until the user confirms resolution ("thumbs up").
5. The resolved session — including the working configuration path — is persisted in the user's WR Graph™, so the graph afterwards holds all details of the successful setup.

Representative use case: fully automated execution of complex third-party configurations (e.g. a Cloudflare setup), where the permission grant covers the configuration steps and any residual failure state is round-tripped as screenshot plus description until a tailored workflow succeeds.

I.16 Feedback Reintegration into the WR Graph™ (Normative)

Outcomes of automation runs, support cases, errors, user actions, and website interactions MAY be reintegrated into the WR Graph™ as versioned context, subject to:

- explicit provenance and temporal ordering (consistent with the append-and-enrich model of I.12.2)
- scope binding to the originating relationship, project, or context region
- the organizational layer's background sorting (I.14.4), which assigns reintegrated material to its topic without user effort

Reintegrated feedback SHALL NOT modify capability scopes, permission grants, or enforcement rules; it enriches context only. User feedback MAY, however, cause an automation path to be **superseded or deprecated** within the graph (e.g. after repeated negative ratings or confirmed interface drift), so that routing ceases to surface stale paths; supersession preserves lineage and is driven exclusively by local, user-authorized signals (I.14.5). Over time this yields iterative improvement of the graph per project and per relationship.

I.17 WCR Authoring Lifecycle (Normative)

I.17.1 Authoring Scope and Storage Targets

WCR authoring is not restricted to publishers. Any user MAY author recordings for their own environment; publishers author them for delivery through a BEAP™ Gateway (I.15.6). A confirmed recording is stored in two places:

- the author's **recording library**, where it is versioned and retrievable as an artifact in its own right, and
- where meaningful, the **WR Graph™**, embedded into the appropriate domain region as an executable automation path — assignment is performed automatically by the organizational layer (I.14.4), not by manual filing.

I.17.2 Authoring Paths

A Workflow Context Recorder artifact MAY be authored via two paths:

Path A — Video-derived authoring. The user uploads a video (screen recording, tutorial, or third-party material such as a YouTube video) together with a textual instruction

describing the intended workflow. The system derives a candidate semantic manuscript from video and instruction.

Path B — Direct recording. The user performs the workflow while the recorder captures all available signals simultaneously — video, audio, the semantic action sequence, and structural interface metadata — and fuses them into one accurate semantic instruction. No single signal is authoritative; fusion follows the precedence rules of I.11.6.

I.17.3 Validation Loop

Authoring SHALL proceed through a staged validation loop:

1. **Candidate manuscript** is generated (Path A or B).
2. **Supervised test run:** the candidate is executed in the sandbox; the semantic layer of the execution is recorded and reconciled against the manuscript.
3. **Final test run:** the reconciled manuscript is executed again for confirmation.
4. **User confirmation:**
    - **Thumbs up** → the manuscript is promoted to a stored WCR record and automatically classified into the appropriate topic region of the WR Graph™ by the organizational layer (background sorting, no manual filing).
    - **Thumbs down** → the user describes what went wrong; the description enters the manuscript as corrective context, and the loop returns to step 2.

I.17.4 Preserved Constraints

All constraints of I.6 apply unchanged to both authoring paths: no executable macros, no replay scripts, no binding to static selectors or pixel coordinates; the stored artifact is a structured textual manuscript requiring mediated interpretation under BEAP™ governance.

Video-derived candidates (Path A) SHALL be treated as external-origin artifacts and remain confined to the sandbox trust domain until validated (I.2.2, I.3).

I.17.5 Tool Invocation Constraints (Normative)

WCR recordings encompass tool use, but tool use SHALL be treated as an explicit, separately governed element of the recording — not as an implicit side effect of replay:

- Every tool invocation within a recording SHALL be declared in the manuscript with tool identity and declared inputs.
- Tools SHALL be subject to **explicit policy whitelisting**. Invocation of a non-whitelisted tool SHALL fail closed; whitelisting is a policy-engine decision, never a routing outcome.
- Tool invocations SHALL be routed through the sandbox sub-orchestrator unless a tool is explicitly designated for host execution by policy.
- Tool results SHALL be returned to the calling context in **text format only**, consistent with the text-purity invariant; binary or opaque results remain confined to the sandbox trust domain.
- Once a recording is finalized, its tool invocations are part of the deterministic execution artifact (I.14.5): fixed, inspectable, and cryptographically bound at the point of consequence.

Terminology

WRVault™ (Normative)

WRVault™ denotes a locally controlled, encrypted, multi-layered secure vault bound to executing identity and environment.

Data SHALL be:

- Compartmentalized
- Scope-restricted
- Purpose-restricted
- Cryptographically isolated by class
- Exposed only through controlled augmented contexts

License Notice

This Annex forms an integral part of the WR Desk™ specification and is licensed under the same terms as BEAP™, WR Desk™, and PoAE™.